

novas_dnd.h Reference Manual

File Name

novas_dnd.h

Description

This file contains the function prototypes needed to interface with Novas' Drag and Drop interface for Motif applications. Using this library it is possible to drag signals from Debussy and drop them into another application. The reverse situation is also possible.

The Drag/Drop atom names for target types are defined in the following string macros. These can be passed to `XmInternAtom()`.

NOVSDND_SIGNAL_ATOM_NAME, *NOVSDND_INST_ATOM_NAME*,
NOVSDND_SOURCE_ATOM_NAME.

Examples

Consider the case where 3 signals are dragged from *nWave* to another 3rd party tool (Motif application).

3rd party tool, gets a drop callback for *NOVSDND_SIGNAL_ATOM_NAME*, the drop string is placed in *motif_drop_string*.

```
pDnd pDrop;
int  nNumSignals;
char *signal;

pDrop = DropnovasDropNew(NOVSDND_SIGNAL, ",", False, NOVSDND_VERILOG);
novasDropSetDropStr(pDrop, motif_drop_string);
nNumSignals = novasDropGetDropNum(pDrop);
for (int i = 0; i < nNumSignals; i++)
{
    if (novasDropGetSignal(pDrop, i, &signal))
    {
        signal = strdup(signal);
        ...
    }
}

novasDropFree(pDrop);
```

* * *

3rd party tool prepares 2 signals to drag to *nWave*.

```
pDnd pDrag;  
char *dragString;
```

```
pDrag = novasDragNew(NOVASDND_SIGNAL, ".", False, NOVASDND_VERILOG);  
novasDragAddSignal(pDrag, "top.i_cpu.clock");  
novasDragAddSignal(pDrag, "top.i_cpu.reset");  
novasDragGetStr(pDrag, &dragString);
```

3rd party tools passes dragString to Motif for NOVASDND_SIGNAL_ATOM_NAME

...

```
novasDragFree(pDrag);
```

Library-name

libndna.a and *libnsys.a*

Data Type Definitions

pDnd

```
typedef struct ulDropStr2* pDnd;
```

novasdndLanguage

```
enum { NOVASDND_VERILOG , NOVASDND_VHDL }
```

novasdndType

```
enum { NOVASDND_SIGNAL , NOVASDND_INST , NOVASDND_SOURCE }
```

Functions

- `pDnd novasDragNew(nType, strDelimiter, bRootDeli, nLanguage)`
- `void novasDragFree(dragObj)`
- `bool_T novasDragAddSignal(dragObj, cstrSignal)`
- `bool_T novasDragAddInst(dragObj, cstrInst)`
- `bool_T novasDragAddSource(dragObj, cstrInst, cstrFilename, nBeg, nEnd)`
- `int novasDragGetStrLen(dragObj)`
- `void novasDragGetStr(dragObj, pstr)`
- `pDnd novasDropNew(nType, strDelimiter, bRootDeli, nLanguage)`
- `void novasDropFree(dropObj)`
- `bool_T novasDropSetDropStr(dropObj, cstrDrop)`
- `char* novasDropGetDelimiter(dropObj)`
- `int novasDropGetDropNum(dropObj)`
- `bool_T novasDropGetSignal(dropObj, nNo, pstrSignal)`
- `bool_T novasDropGetInst(dropObj, nNo, pstrInst)`
- `bool_T novasDropGetSource(dropObj, nNo, pstrInst, pstrFilename, pnBeg, pnEnd)`
- `bool_T novasDragAddSignalDeclInfo(pDnd dragObj, const char* cstrSignal, const char* cstrDeclType, const char* cstrPackedRange, const char* cstrUnpackedRange, char bDecl);`

pDnd novasDragNew(nType, strDelimiter, bRootDeli, nLanguage)

<i>Parameters</i>	• <i>novasdndType nType;</i> The type of drag object. This can be NOVASDND_SIGNAL, NOVASDND_INST or NOVASDND_SOURCE • <i>char* strDelimiter;</i> The delimiter to be used for separating instance / signal elements, example "." • <i>bool_T bRootDeli;</i> True if a root delimiter is to be used, example .top.system.i_cpu • <i>novasdndLanguage nLanguage;</i> What HDL language is being used. Can be one of NOVASDND_VERILOG or NOVASDND_VHDL
<i>Description</i>	Construct a new drag object.

<i>Return Value</i>	<i>pDnd</i> Returns a pointer to a drag object
<i>Examples</i>	<pre>pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG);</pre>

void novasDragFree(dragObj)

<i>Parameters</i>	<i>pDnd dragObj;</i> The drag object to be freed. Must have been allocated by novasDragNew().
<i>Description</i>	Free a drag object previously allocated with novasDragNew()
<i>Return Value</i>	<i>void</i>
<i>Examples</i>	<pre>pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); ... novasDragFree(pDragObject);</pre>

bool_T novasDragAddSignal(dragObj, cstrSignal)

<i>Parameters</i>	<i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). <i>const char* cstrSignal;</i> Signal name
<i>Description</i>	Add a signal into this drag object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.
<i>Examples</i>	<pre>pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); ... novasDragAddSignal(pDragObject, "i_cpu");</pre>

bool_T novasDragAddInst(dragObj, cstrInst)

<i>Parameters</i>	<i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). <i>const char* cstrInst;</i> Instance name.
-------------------	--

<i>Description</i>	Add an instance into this drag object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.
<i>Examples</i>	<pre>pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); ... novasDragAddInst(pDragObject, "top.system.i_cpu");</pre>

bool_T novasDragAddSource(dragObj, cstrInst, cstrFilename, nBeg, nEnd)

<i>Parameters</i>	• <i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). • <i>const char* cstrInst;</i> Instance name • <i>const char* cstrFilename;</i> Filename of code block. • <i>int nBeg;</i> The starting line of code block. 1-base • <i>int nEnd;</i>
<i>Description</i>	Add a source code block into this drag object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.
<i>Examples</i>	<pre>pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); ... novasDragAddInst(pDragObject, "top.system.i_cpu", "myfile", 1, 20);</pre>

int novasDragGetStrLen(dragObj)

<i>Parameters</i>	• <i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew().
<i>Description</i>	Used to get the length of the string returned by novasDragGetStr().
<i>Return Value</i>	<i>int</i> The length of the string returned by novasDragGetStr().

<i>See Also</i>	novasDragGetStr()
-----------------	-------------------

void novasDragGetStr(dragObj, pstr)

<i>Parameters</i>	• <i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). • <i>char** pstr;</i> Will point to the string to be used in drag operations.
<i>Description</i>	Get the string representation of the information stored in this drag object. The returned string can then be used as the drag source. Note: The memory for the string is allocated by novasDragGetStr() and will be freed on the next call to this function.
<i>Return Value</i>	<i>void</i>
<i>See Also</i>	novasDragGetStrLen()

pDnd novasDropNew(nType, strDelimiter, bRootDeli, nLanguage)

<i>Parameters</i>	• <i>novasdndType nType;</i> The type of drag object. This can be NOVASDND_SIGNAL, NOVASDND_INST or NOVASDND_SOURCE. • <i>char** strDelimiter;</i> The delimiter to use separate delimiter to be used for separating instance / signal elements. • <i>bool_T bRootDeli;</i> True if a root delimiter is to be used, example .top.system.i_cpu • <i>novasdndLanguage nLanguage;</i> What HDL language is being used. Can be one of NOVASDND_VERILOG or NOVASDND_VHDL.
<i>Description</i>	Construct a new drop object.
<i>Return Value</i>	<i>pDnd</i> Returns a pointer to a drop object
<i>Examples</i>	<pre>pDnd pDropObject = novasDropNew(NOVASDND_SIGNAL, True, NOVASDND_VERILOG);</pre>

void novasDropFree(dropObj)

<i>Parameters</i>	• <i>pDnd dragObj</i>; Pointer to a drag object previously allocated by novasDragNew().
<i>Description</i>	Free a drop object previously allocated with novasDropNew().
<i>Return Value</i>	<i>void</i>
<i>Examples</i>	<pre>pDnd pDropObject = novasDropNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); ... novasDropFree(pDropObject);</pre>

bool_T novasDropSetDropStr(dropObj, cstrDrop)

<i>Parameters</i>	• <i>pDnd dragObj</i>; Pointer to a drag object previously allocated by novasDragNew(). • <i>const char* cstrDrop</i>; Motif drop string.
<i>Description</i>	Set the drop string returned from Motif to this object for extraction of components.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.

char* novasDropGetDelimiter(dropObj)

<i>Parameters</i>	• <i>pDnd dragObj</i>; Pointer to a drag object previously allocated by novasDragNew().
<i>Description</i>	Used to get the delimiter used for this drop object.
<i>Return Value</i>	<i>char*</i> Pointer to delimiter string.

int novasDropGetDropNum(dropObj)

<i>Parameters</i>	• <i>pDnd dragObj</i>; Pointer to a drag object previously allocated by novasDragNew().
<i>Description</i>	Get the number of items in this drop object. For example, how many signals are in this drop.

<i>Return Value</i>	<i>int</i> Number if items in this drop object.
---------------------	--

bool_T novasDropGetSignal(dropObj, nNo, pstrSignal)

<i>Parameters</i>	• <i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). • <i>int nNo;</i> The index of the signal to retrieve. 0 - based. • <i>char** pstrSignal;</i> The will point to the returned signal. Note: The memory for the string is allocated by novasDropGetSignal() and will be freed on the next call to this function.
<i>Description</i>	Gets one signal from this drop object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.

bool_T novasDropGetInst(dropObj, nNo, pstrInst)

<i>Parameters</i>	• <i>pDnd dragObj;</i> Pointer to a drag object previously allocated by novasDragNew(). • <i>int nNo;</i> The index of the signal to retrieve. 0 - based. • <i>char** pstrInst;</i> The will point to the returned signal. Note: The memory for the string is allocated by novasDropGetSignal() and will be freed on the next call to this function.
<i>Description</i>	Gets one instance from this drop object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.

bool_T novasDropGetSource(dropObj, nNo, pstrInst, pstrFilename, pnBeg, pnEnd)

<i>Parameters</i>	• <i>pDnd dragObj;</i>
-------------------	--

	Pointer to a drag object previously allocated by novasDragNew(). • <i>int nNo;</i> The index of the signal to retrieve. 0 - based. • <i>char** pstrInst;</i> The will point to the returned instance. Note: The memory for the string is allocated by novasDropGetSource() and will be freed on the next call to this function. • <i>char** pstrFilename;</i> The will point to the returned source filename. Note: The memory for the string is allocated by novasDropGetSource() and will be freed on the next call to this function. • <i>int* pnBeg;</i> Pointer to an integer which will be set to the beginning line number of the block of source code. • <i>int* pnEnd;</i> Pointer to an integer which will be set to the ending line number of the block of source code.
<i>Description</i>	Gets one source description from this drop object.
<i>Return Value</i>	<i>bool_T</i> True on success; otherwise, False.

bool_T novasDragAddSignalDeclInfo(pDnd dragObj, const char* cstrSignal, const char* cstrDeclType, const char* cstrPackedRange, const char* cstrUnpackedRange, char bDecl);

<i>Parameters</i>	• <i>pDnd dragObj;</i> The drag object to be freed. Must be allocated by novasDragNew(). • <i>const char* cstrSignal;</i> Full signal hierarchy path without range. • <i>const char* cstrDeclType;</i> The signal type without including the direction type (ex: input, output, inout, ref) • <i>const char* cstrPackedRange;</i>
-------------------	--

	Packed array information. <code>const char* cstrUnpackedRange</code> Unpacked array information. <code>char bDecl;</code> Specify whether the dragged signal is the declaration.
<i>Description</i>	Add dragged signal declaration information.
<i>Return Value</i>	<code>bool_T</code> Returns TRUE/FALSE
<i>Examples</i>	<pre> pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); const char* cstrSignal="data"; const char* cstrDeclType="wire"; const char* cstrPackedRange="[7:0]"; //if no range, set "NULL" const char* cstrUnpackedRange="NULL"; //if no range, set "NULL" char bDecl; bool_T bGetinform=novasDragAddSignalDeclInfo(pDragObj ect, cstrSignal, cstrDeclType, cstrPackedRange, cstrUnpackedRange, bDecl) </pre>
<i>Note</i>	If no range, set "NULL" string for <code>cstrPackedRange</code> and <code>cstrUnpackedRange</code> .

`bool_T novasDropGetDeclInfo(pDnd dropObj, int nNo, char** pstrSignal, char** pstrDeclType, char** pstrPackedRange, char** pstrUnpackedRange, char *pbDecl);`

<i>Parameters</i>	<code>pDnd dragObj;</code> The dragged object to be freed. Must be allocated by <code>novasDragNew()</code>. <code>int nNo;</code> Signal index to get signal. <code>const char* cstrSignal;</code> Full signal hierarchy path without range. <code>const char* cstrDeclType;</code> The signal type without including the direction type (ex: input, output, inout, ref)
-------------------	--

	• <code>const char* cstrPackedRange;</code> Packed array information. • <i><code>const char* cstrUnpackedRange</code></i> Unpacked array information. • <i><code>char bDecl;</code></i> Specify whether the dragged signal is the declaration.
<i>Description</i>	Get drag signal declaration information.
<i>Return Value</i>	<i>bool_T</i> Returns TRUE/FALSE
<i>Examples</i>	<pre> pDnd pDragObject = novasDragNew(NOVASDND_SIGNAL, ".", True, NOVASDND_VERILOG); int nNo=0, const char* cstrSignal; const char* cstrDeclType; const char* cstrPackedRange; const char* cstrUnpackedRange; char bDecl; bool_T bGetinform= novasDropGetDeclInfo (pDragObject, nNo, cstrSignal, cstrDeclType, cstrPackedRange, cstrUnpackedRange, bDecl) </pre>
<i>Note</i>	if no range, <code>cstrPackedRange</code> and <code>cstrUnpackedRange</code> are "NULL" string.